



MONASH University

FIT2099 A2 DESIGN RATIONALE

CL_Lab08TeamC

Kian Lok Chin	35090294
Lim Chi Jian	35040424
Kar May Ong	34477241
Pemudi Hiruni Halgahawatta	34246517
Liyanaarachch	
Youssef Ahmed Adel Attia	34073817
Hassanein	

REQ1: The Life in the Valley

A. Creature Production

Design 1: In-class reproduction logic aka Implement offspring spawning directly in SpiritGoat class.

Pros	Cons
Straightforward: no additional classes required.	Reproduction logic intermingles with movement, reducing clarity and testability.
	Modifications to reproduction triggers require changing SpiritGoat, breaking Open/Closed Principle

Design 2: Producible interface with ProduceBehaviour

Pros	Cons
Encapsulates reproduction in ProduceBehaviour, adhering to Single Responsibility.	Introduces interface and behaviour classes, adding initial complexity.
Facilitates extension: new producers need no changes to SpiritGoat.	

B. Egg Implementation

Design 1: Creating specific egg classes

Pros	Cons
Avoids the "fat interface" problem, i.e., inheriting too many unnecessary methods	Increased code duplication; shared functionalities must be re-implemented in each egg class
Avoids unnecessary constraints that inheritance may introduce, such as the hatching mechanism	Difficult to implement unified egg-related functionalities, such as a common hatching mechanism
	Difficult to manage all egg types uniformly, e.g., cannot use polymorphism effectively

Design 2: Produce Different Eggs via Passed-in Parameters

Pros	Cons
Uses the Condition interface to abstract and support multiple hatching conditions.	Memory usage may grow with many Egg instances.
Enables adding new hatching conditions and effects without changing existing code.	The number of parameters passing through Egg instance is long.
Promotes code reuse by configuring Egg instances with different parameters.	

C. Egg Consumption Mechanism

Design 1: Generic ConsumableItem base to handle eating effects.

Pros	Cons
Reuses consumption logic for all edible items.	May overgeneralize effects, forcing items to fit a uniform model.
Keeps effect application consistent.	Unique item effects can complicate the base class.

Design 2: Eatable interface and ConsumptionEffect composite.

Pros	Cons
Provides a unified abstraction for edible behaviour via the Eatable interface	It can be harder to trace and debug the effect chain.
Uses Composite Pattern List<Effect> to combine multiple consumption effects.	Composite pattern may add unnecessary complexity for simple cases.

D. AdjacentCapabilityChecker Implementation

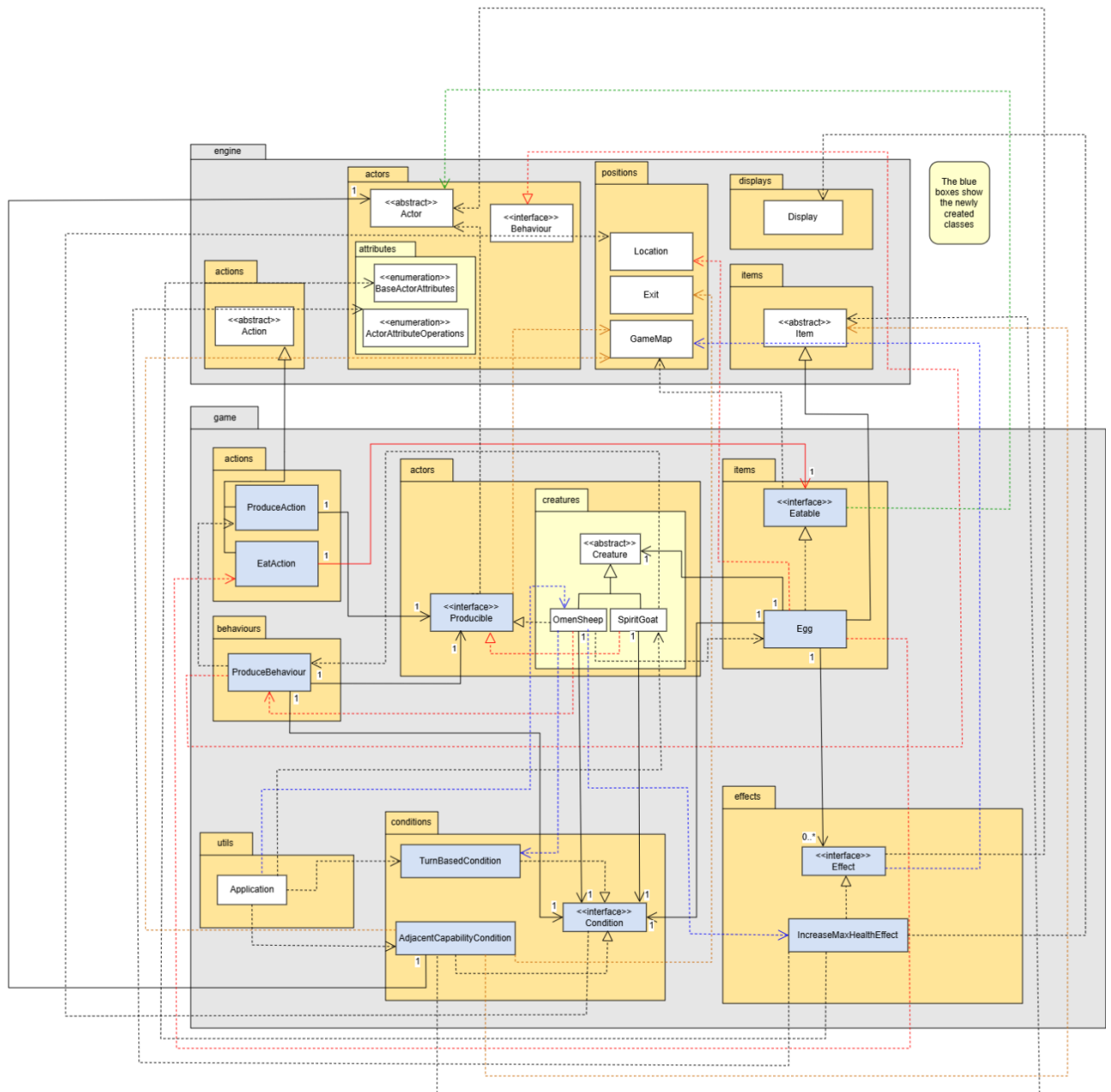
Design 1: Add a Condition (AdjacentCapabilityCondition) class.

Pros	Cons
Centralized code, easy to locate	Change in checking logic require additional classes.
Reduce code duplication and promotes DRY	
Able to adapt to different capabilities/enumeration in the future such as POISONED.	

Design 2: Embed checking logic directly in methods Implement adjacency checking logic directly in methods that require it.

Pros	Cons
Simplifies immediate implementation	Violates DRY principle through code duplication
No additional classes to manage	Changes to checking logic require updates in multiple places
Direct access to local context	Reduced testability as logic is scattered

REQ1 - UML Diagram



The above diagram shows the final design of Req1 in which we chose Design 2 for **Creature Production, Egg Implementation** and **Consumption Mechanism** whereas we use Design 1 for **AdjacentCapabilityChecker** implementation.

The architecture we've developed strictly adheres to SOLID principles, which provides significant benefits for maintainability and extensibility. For instance, the Single Responsibility Principle (**SRP**) is demonstrated through our clear separation of concerns: ProduceBehaviour and ProduceAction exclusively manages produce behaviours and EatAction handles consumption effects.

This separation prevents any single class from becoming a "god class" with too many responsibilities, making the codebase easier to understand, test, and maintain.

Consider a hypothetical scenario where we need to modify how creatures reproduce. In a non-SRP design, we would need to modify the SpiritGoat class directly, potentially introducing bugs in unrelated functionality. Our implementation isolates reproduction logic in ProduceBehaviour and ProduceAction, allowing targeted changes without disturbing movement or rendering code.

The Open/Closed Principle (**OCP**) is respected through our behavioural injection approach with the Producible and Eatable interfaces. When a new creature type needs to be added, we can simply implement these interfaces without modifying existing framework code. For example, if requirements change to include a new "PhoenixBird" that reproduces differently, we can make it implement the Producible interface and implement its own produce method without changing a single line of code in the core framework.

The Interface Segregation Principle (**ISP**) is reflected in our carefully scoped interfaces. For example, Producible exposes only methods related to production capabilities, while Eatable focuses exclusively on consumption behaviours. This prevents the "fat interface" problem where implementers are forced to provide irrelevant functionality.

The Dependency Inversion Principle (**DIP**) is evident in how high-level modules like the game engine and action scheduler depend on abstractions (ProduceBehaviour, EatAction) rather than concrete implementations. This inversion allows the engine to remain agnostic about specific producers or consumables, enhancing system flexibility and testability. For instance, the game engine doesn't need to know whether a SpiritGoat or OmenSheep is producing offspring as it simply works with the Producible interface. Instead of tying the system to specific subclasses like a parameterized Egg class, it encapsulates shared lifecycle logic. Specific behaviours are injected via parameters or strategies, keeping the system flexible, decoupled, and easy to extend without altering core logic, enhancing maintainability and scalability.

We've also prioritized the Don't Repeat Yourself (**DRY**) principle throughout our design. By centralizing adjacency checking in the AdjacentCapabilityChecker utility class rather than embedding this logic in various methods, we've eliminated potential duplication and ensured consistent behaviour. Instead of separate subclasses like OmenSheepEgg or

GoldenBeetleEgg, a single parameterized Egg class handles common lifecycle functions. Specific behaviours are set via instantiation parameters, reducing code duplication and improving flexibility by enabling new egg types without new subclasses.

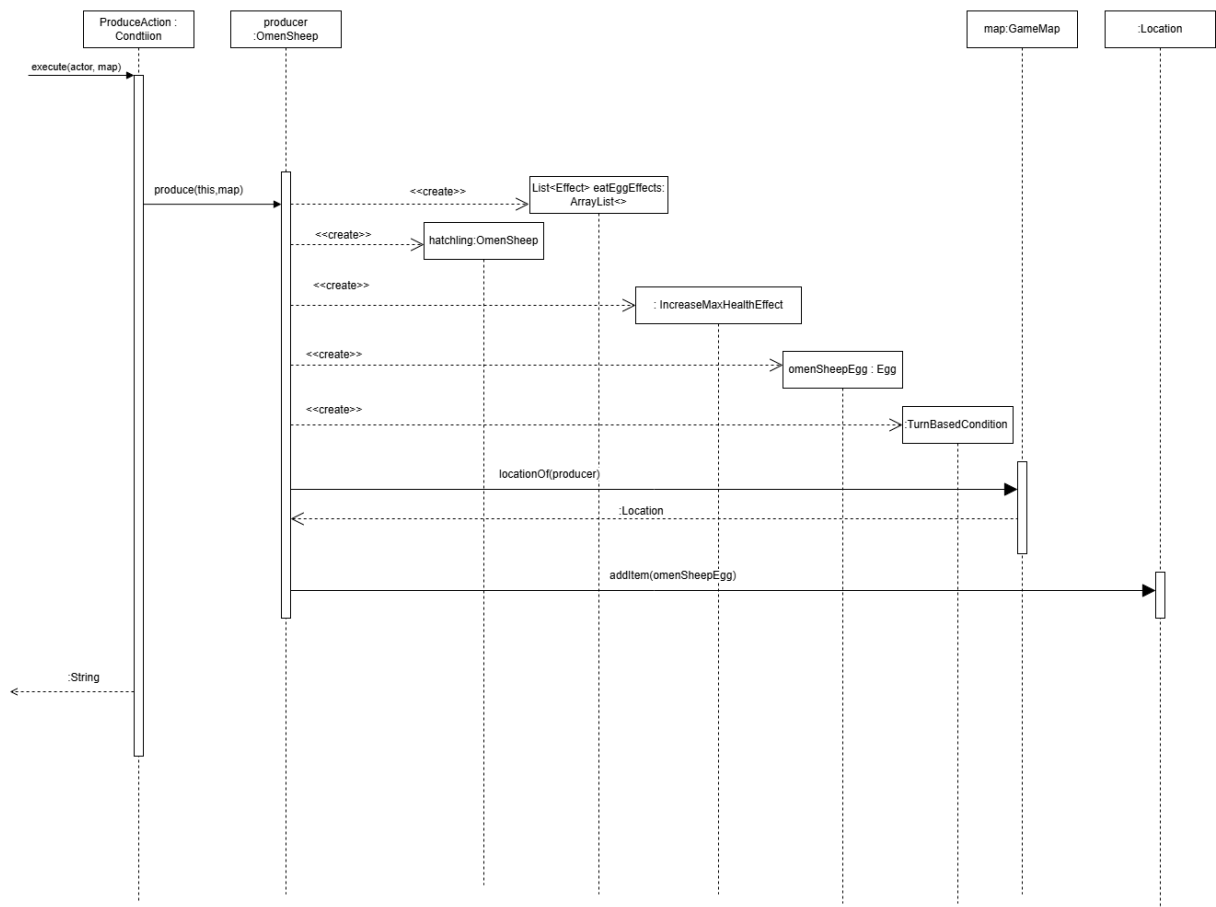
From a connascence perspective, our design carefully manages various coupling types to enhance maintainability. It addresses Connascence of Name through consistent method naming conventions across interfaces and implementation classes. Connascence of Algorithm and Type is managed through interfaces that enforce specific method signatures and return types, ensuring uniform interaction patterns. Connascence of Timing and Value is handled through well-parameterized configurations, making behaviour execution predictable.

If new requirements emerge, such as adding new creature types or changing reproduction mechanics, our design ensures minimal impact on existing systems. For example, if business requirements change to make reproduction dependent on time of day, we would only need pass in the produce condition to the creature such as OmenSheep and SpiritGoat which makes it easy extensible.

Our design comprehensively addresses all requirements while establishing a robust foundation that anticipates future changes. The architecture provides clear extension points for new features while maintaining a cohesive overall structure, balancing immediate implementation needs with long-term maintainability

REQ1 – Sequence Diagram

This sequence diagram is specific to the scenario of OmenSheep's egg production and the effects of interacting with the egg



REQ 2: Be wary of dung (beetle)

A. GoldenBeetle Implementation

Design 1: Create GoldenBeetle class extending Creature abstract class and implement Eatable and Producible interfaces

Pros	Cons
Promotes code reusability	Requires maintenance for extra class and interfaces
Clear modularisation of responsibilities using interfaces and abstract classes	Results in complex program if every functionality needs abstract classes and interfaces
Enhances code readability as the abstract class and interfaces define a clear contract of what is expected for GoldenBeetle class	

Design 2: Implement all logic directly inside the GoldenBeetle class

Pros	Cons
Enhances code maintainability as no additional classes or interfaces need to maintain	GoldenBeetle class takes on multiple responsibilities
	Existing code need modification if new behaviours are added

B. Following Behaviour

Design 1: Directly implement getAction() and distance() methods in GoldenBeetle class

Pros	Cons
Enhances code maintainability as no additional class need to be used	Class is responsible for both its behaviour and following logic
Simpler implementation if only GoldenBeetle can follow an actor	Duplicated code will be implemented across every actor with the ability to follow a followable actor

Design 2: Create a FollowBehaviour class

Pros	Cons
Promotes code reusability as other actors can easily reuse this class	Requires maintenance of an extra class
Allows runtime modification	Connascence of Meaning – Modification needed if the meaning and implementation of the behaviour change

C. Restore Player's Stamina after Consuming GoldenEgg

Design 1: Parse the parent of the Egg when creating a new Egg, check the parent's type using 'instanceof' and restore player's stamina directly if the identified parent is a GoldenBeetle in the eat() method in Egg class

Pros	Cons
Simpler and direct implementation	Modification needed when a new type of egg is added with different consumption effect
Enhances code maintainability as no additional classes or interfaces need to maintain	Violates Liskov Substitution Principle - Requires checking of type of creature using 'instanceof'

Design 2: Create a new class RestoreStaminaEffect and stores it in Egg's consumption effects to apply the effects when the egg is eaten in the eat() method in Egg class

Pros	Cons
Enhance code extensibility as any new effect can be simply applied by adding the class in the egg's list of consumption effects	Requires maintenance for an extra class
Enhance code reusability as other class that have the same effect can reuse this class	

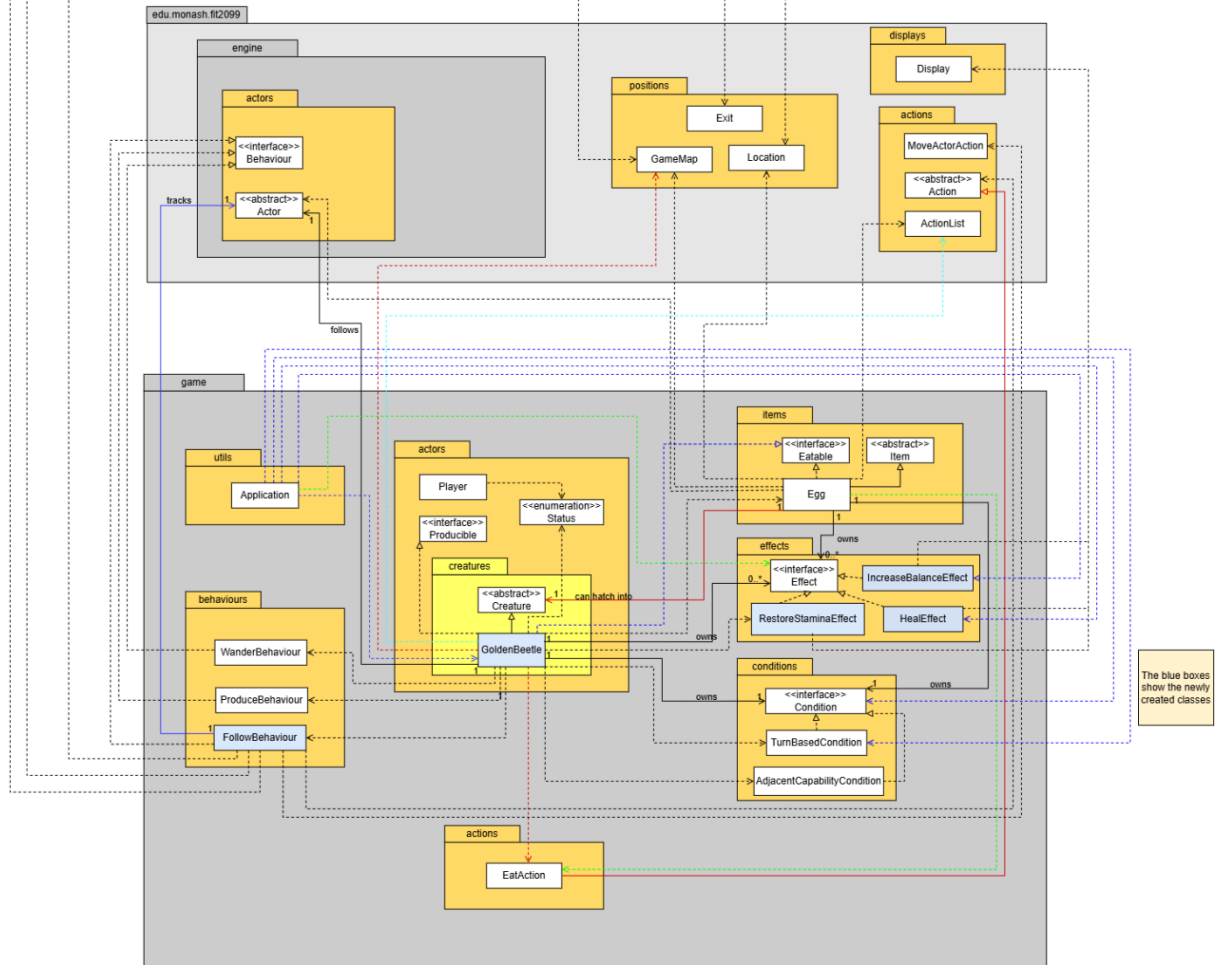
D. Hatching Condition of GoldenEgg

Design 1: Use 'instanceof' and downcasting to check the type of ground that GoldenEgg can hatch on in the shouldHatch() method of Egg class

Pros	Cons
Simpler and direct implementation	Modification needed when a new cursed entity is added
	Violates Liskov Substitution Principle - Requires checking of type of ground using 'instanceof'

Design 2: Create a new class AdjacentCapabilityCondition that utilises Status enum class to check if GoldenEgg can be hatched (located near CURSED entities)

Pros	Cons
Enhance code extensibility as the status can be added to future cursed entity that allow GoldenEgg to be hatched nearby it	Additional status logic implementation in the ground classes needed
Enhance code readability	



The diagram above shows the final design of REQ2, which utilizes Design 1 for **GoldenBeetle implementation**, Design 2 for **following behaviour**, **restoration of stamina after consuming a GoldenEgg** and **hatching condition check of GoldenEgg**.

The GoldenBeetle class extends the abstract Creature class. The new concrete class inherits the methods and attributes from its existing abstract class. We abstract these identities to avoid repetition in code (DRY) as this class shares many same attributes and methods with the abstract class it extends from. The new creature created solely uses the constructor of its super class with different initialized values, enabling it to replace the usage of their subclass in code (Liskov Substitution Principle).

In the GoldenBeetle class, the allowableActions() is overridden to add an eat action and follow behaviour when it's nearby an Followable actor, giving GoldenBeetle class the responsibility of adding an eat action to the options, reducing dependencies of other classes on this class but segregating this responsibility (Single Responsibility Principle).

Besides, GoldenBeetle class relies on abstraction rather than concrete implementation, adhering Dependency Inversion Principle. Moreover, using different interfaces to define different traits (i.e. can produce) and abilities (i.e. ability to spawn and be eaten) of the concrete GoldenBeetle class allows us to separate different responsibilities to different interfaces, resulting in better maintainability of the system (Single Responsibility Principle). This implementation allows the GoldenBeetle's traits to be separated by their own purposes (Interface Segregation Principle). Future actions and behaviours can simply be added or implemented by the GoldenBeetle class, without changing the existing classes, adhering to the Open Closed Principle.

Adding a new FollowBehaviour allows us to segregate the logic of tracking the movement of an actor and moving following the target from the GoldenBeetle class itself, rather than embedding the following logic directly in the GoldenBeetle class by implementing getAction() and distance(). Using this implementation, the GoldenBeetle class will be responsible for its own purpose only, where different responsibilities are distributed to different classes (Single Responsibility Principle). As this design adheres to the Open Closed Principle, there will be minimal impact on the existing class if any new requirement for the following behaviour arises (e.g. adding logic to only follow the target when it is within a specific range in FollowBehaviour class). The priority of behaviours is managed using an order-based TreeMap.

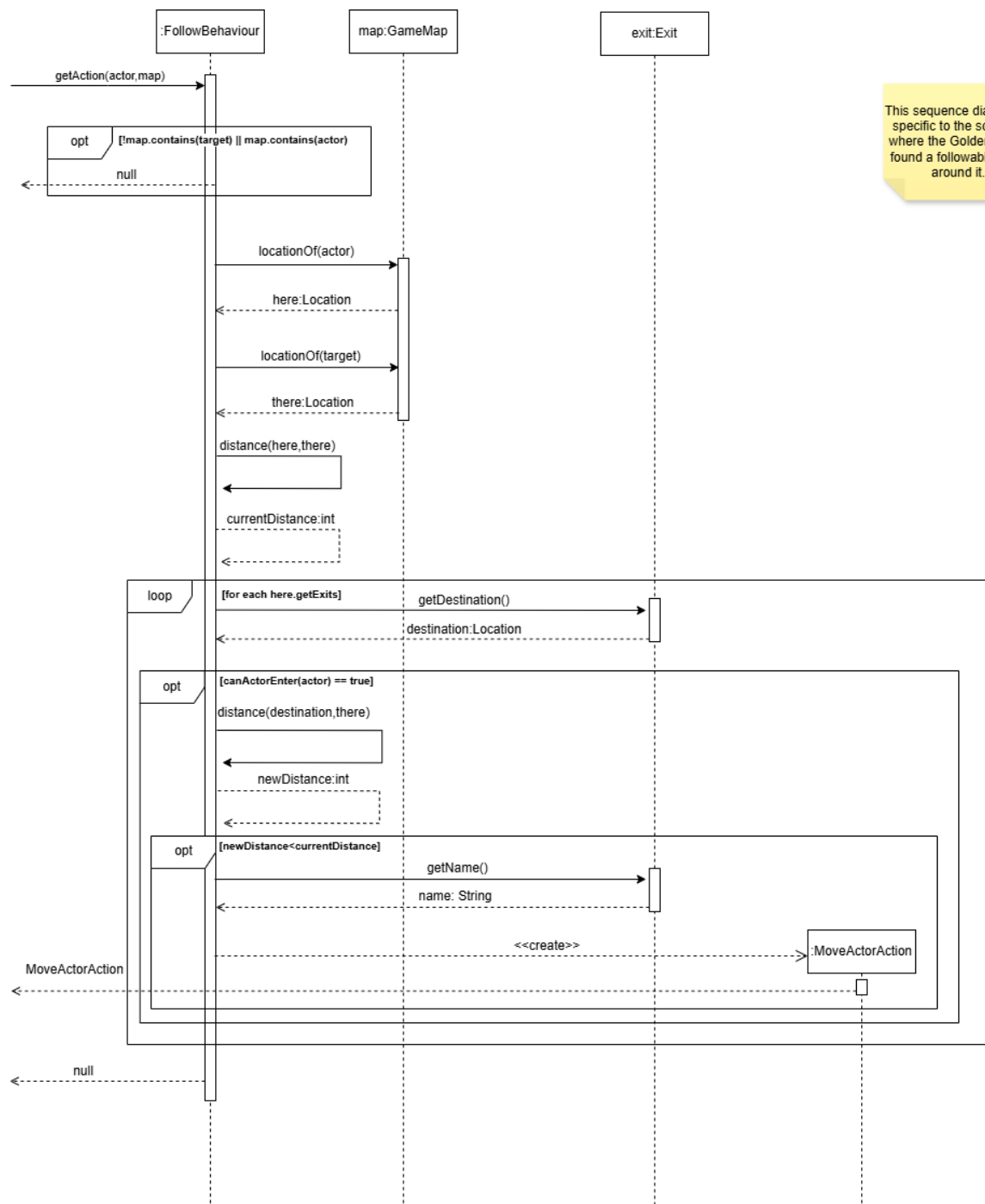
Creating a new class RestoreStaminaEffect enhances code reusability as this effect can be shared across different classes without duplicating the logic (DRY). We also avoided using 'instanceof' to check the parent of the egg and apply consumption effects based on the parent identified, adhering to the Liskov Substitution Principle. Moreover, this design adheres to the Open Closed Principle by allowing new consumption effects to be introduced and added (e.g. HealEffect class can be easily added to the list of consumption effects of the creature) without modifying the existing code.

Egg class will depend on Status enum class, which acts as an indicator to allow GoldenEgg to be hatched when it is nearby cursed entities. Using the dependency on the Status enum class, we avoided using downcasting to check for the types of the concrete classes that are fine to be hatched on, resulting in easier functionality extension without modifying existing code with additional if-else checks by allowing new ground to have CURSED status (Open-Closed Principle).

While connascence cannot be entirely eliminated, this final design has reduced the severity of connascence. Using interfaces and abstract classes in GoldenBeetle class, the GoldenBeetle class have implemented all required methods and adhered to the methods' signature (Connascence of Name). Besides, in the Egg class, the Egg depends on Condition, Creature and Effect classes (which are all abstract classes), making it extensible and minimising changes when those implementations evolve. For example,

when new condition, creature or effect is introduced, no modification is needed for Egg class due to the use of polymorphism (Connascence of Type).

REQ2 – Sequence Diagram



This sequence diagram is specific to the scenario where the GoldenBeetle found a followable actor around it.

REQ 3: The People of the Valley

A. NPC Implementation

Design 1: Create NPC-specific classes that inherit directly from Actor.

Pros	Cons
Each NPC can implement logic specific to its needs without constraints from a base class.	Common logic must be re-implemented in each NPC class.
NPCs with minimal or no monologue logic can avoid inheriting unnecessary functionality.	Adding new shared features would require modifying every NPC class instead of a single base class.
The class hierarchy is flattened, reducing structural complexity.	The lack of a shared abstraction for conversational NPCs makes the system less modular and harder to extend.

Design 2: Create an abstract class with shared logic for NPCs, which the NPCs inherit from.

Pros	Cons
The abstract ConversationalActor class makes it easy to create new NPCs by reusing shared features.	Similar NPC behaviours need to be updated individually, which is time consuming and error-prone.
Monologues can be added with or without conditions, supporting context-sensitive dialogue.	New NPCs add only the behaviours they need, but shared behaviours may lead to duplicated code, increasing maintenance effort.
Changes to the listening logic only need to be made in one place, simplifying debugging and updates.	If some NPCs need specialized listening logic, the base class may become too generic to meet their needs.
The abstract initMonologues() method provides a clear extension point for adding NPC-specific dialogue.	

B. Monologue Implementation

Design 1: Composite Condition Pattern (Monologue Class)

Pros	Cons
Supports diverse checks (money, inventory, attributes) via the Condition interface.	More classes/interfaces to maintain, increasing complexity.
Add new condition types without changing core logic.	

Design 2: Embed monologue in conversational actor class

Pros	Cons
All monologue-related logic is centralized within the ConversationalActor, reducing the need for additional classes.	Monologue logic is tightly coupled with the actor, making it harder to reuse or extend for other types of actors.
Reduces the number of classes and interactions, simplifying the overall system.	If other actors need monologues, the logic must be duplicated or refactored.
	As monologues and conditions increase, the ConversationalActor class may become bloated and harder to maintain.

C. Condition Implementation

Design 1: Handle all conditions in respective class

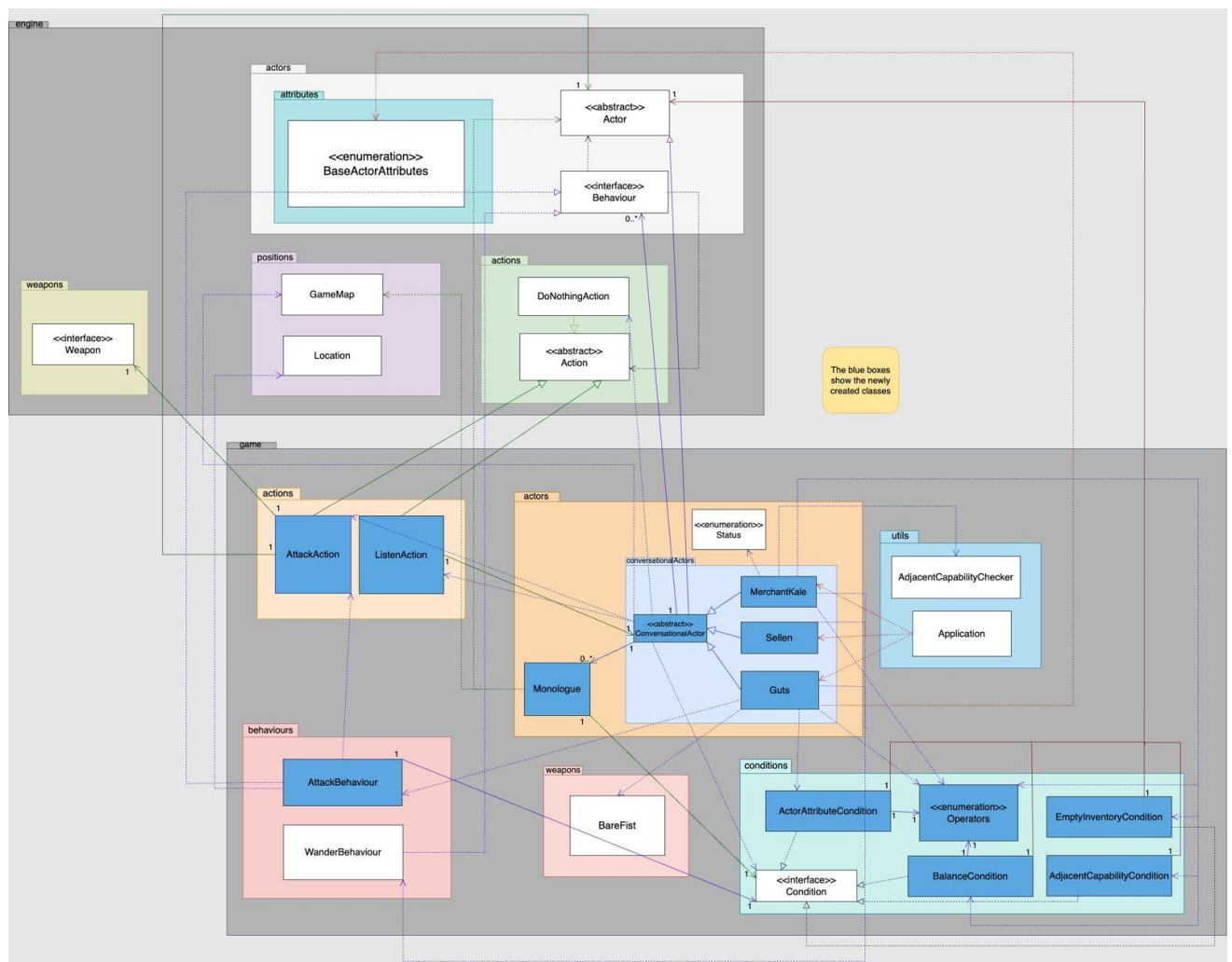
Pros	Cons
Error at a class which uses the condition logic can be found easily	Violate DRY if condition logic are repeated at different classes
No need to create a new class for each new condition.	One class may get messy with lots of if/else or switch statements.
	As conditions grow, the class may become tangled and error-prone.

Design 2: Dedicated Condition Classes

Pros	Cons
Each class has a clear purpose and is easy to understand	Adding a new condition requires creating a new class
Can provide specialized logic for specific conditions	Difficult to manage all condition classes uniformly

Error isolation; problems in one class don't affect others	
Reusable in different contexts	

REQ3 - UML Diagram



The diagram above shows the final design of requirement 3, which utilizes Design 2 for NPC and condition implementation, Design 1 for monologue implementation.

Each class adheres strictly to the **Single Responsibility Principle** with `ConversationalActor` handling only dialogue flow and monologue selection, `Monologue` encapsulating its text and the trigger `Condition`, `ListenAction` displaying speech, NPC subclasses like `Sellen`, `MerchantKale` and `Guts` defining only their own names, stats, behaviours and monologues, and `AttackBehaviour` scanning nearby locations for enemies and when one is found creating an `AttackAction` using the actor's weapon.

The `Condition` interface and its integration with the `Monologue` class create a flexible, maintainable, and extensible dialogue system that follows the **Open Closed Principle**. With a standardized interface, new condition types can be added without changing existing code. `Monologue` evaluates conditions through this interface, keeping logic decoupled and easy to extend. Conditions are reusable, composable, and can be assigned dynamically. NPCs can define custom dialogues in `initMonologues()` without modifying the base `ConversationalActor`, keeping the system stable as it evolves.

All subclasses adhere to the **Liskov Substitution Principle**, so any `ConversationalActor` reference in client code seamlessly accepts instances of `Sellen`, `MerchantKale`, or `Guts`. Each override of `initMonologues()` and every custom behaviour respects the contracts defined in the abstract base, ensuring no unexpected side-effects or broken interactions when substituting one NPC type for another.

The **Interface Segregation Principle** is reflected in narrowly scoped interfaces. The `Condition` interface provides only what it needs, avoiding “fat” interfaces that would force classes to implement unused methods. This lightweight interface design reduces coupling and enhances clarity.

The dialogue system design adheres to the **Dependency Inversion Principle** by having high-level modules depend on the `Condition` abstraction rather than concrete implementations. Both `Monologue` and `ConversationalActor` interact solely with this interface, decoupling core dialogue logic from specific conditions. Concrete condition classes implement the interface, enabling independent evolution without impacting higher-level components. This separation fosters flexible condition composition, reuse across NPCs, and easy integration of new behaviours.

Duplication is minimized via the **Don't Repeat Yourself** principle by placing shared logic in `ConversationalActor` and NPC-specific data in subclasses. The design balances inheritance for NPC hierarchy with composition (e.g., maps/lists) for managing monologues and behaviors. Following **Composition Over Inheritance**, new monologue-triggering logic can be added with minimal disruption. Conditions enable flexible, context-aware dialogue without cluttering classes.

From a **connascence** perspective, our design enhances maintainability by managing key coupling types. It ensures consistent method names (**Connascence of Name**) via the `Condition` interface and `BalanceCondition` class. Interfaces enforce method signatures and return types to handle **Connascence of Algorithm and Type**. The `Operators` enum replaces primitive values to

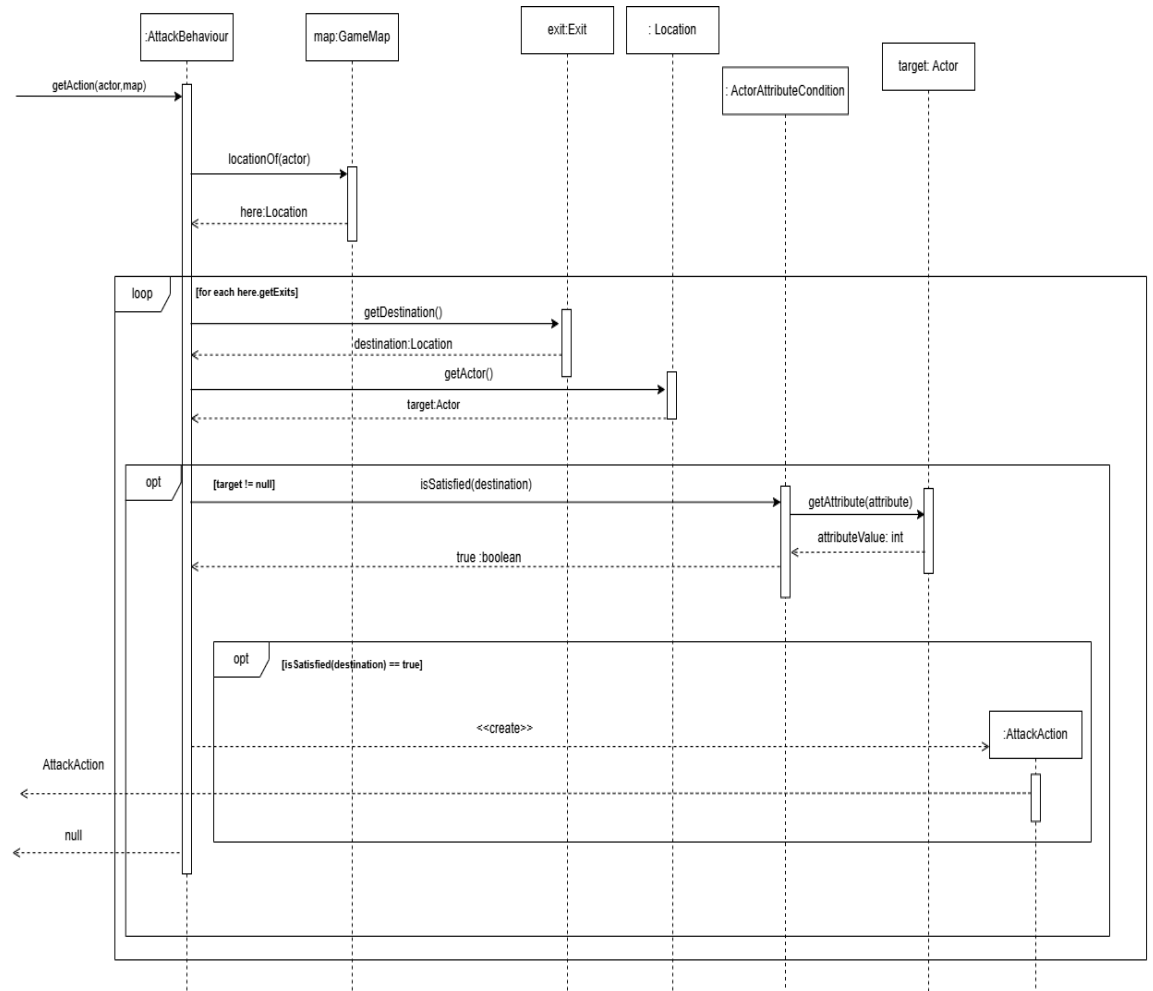
reduce Connascence of Value, while constructor parameters enable logic changes without code edits, minimizing **Connascence of Timing and Value**.

Using the Condition interface to add new dialogue conditions via new classes, avoiding changes to existing code. Composite Condition enables combining triggers flexibly. For example in the future we might have StaminaCondition class that helps check the stamina of the actor, we can simply create it and implement the Condition interface where we can reuse it in the future. New listenable NPCs are extended by subclassing ConversationalActor and mixing in behaviours without altering others as the monologue logic is handled in the ConversationalActor abstract class. Furthermore, new monologues can be added to the respective ConversationalActor by using addMonologue() inside the initMonologue method.

By combining inheritance for NPC hierarchy and composition for behaviours and conditions, the architecture remains modular and maintainable, open for extension but closed to modifying core logic, allowing evolution without losing stability or clarity.

REQ3 – Sequence Diagram

This sequence diagram is specific to the scenario where the Guts attack player/NPCs on its surrounding which has HP lower than 50



REQ4: The Wayward Seller of Wares

A. Merchant Implementation

Design 1: Direct Actor/ConversationalActor Extension

Pros	Cons
Simpler class hierarchy with fewer levels of abstraction	Less separation between merchant functionality and actor functionality
Direct access to Actor methods without intermediaries	More difficult to share merchant-specific behaviour across actors

Design 2: Merchant Interface Implementation

Pros	Cons
Clear separation of concerns between actor and merchant functionality	Slightly more complex with added interface layer
Any actor type can become a merchant via interface implementation	Requires understanding of interface implementation
Promotes loose coupling between actor and merchant roles	
Easier to extend with new merchant types or features	

B. Item Effects Management

Design 1: Direct Effect Implementation in Weapon Classes

Pros	Cons
Simple implementation with effects in weapons	Code duplication across similar weapons
No additional abstractions to manage	Difficult to reuse effects across items
Direct control over each weapon's effects	Changing shared effects needs multiple class updates

Design 2: Separate Effect Classes with Composition

Pros	Cons
Promotes code reuse via composition	More complex architecture with additional classes
Effect logic encapsulated in dedicated classes	Requires understanding of composition pattern
Easy to add or combine effects	
Allows runtime effect composition	

C. Merchant-Item Relationship

Design 1: Direct Item-to-Merchant Binding

Pros	Cons
Simple implementation with direct relationships	Tight coupling between merchants and inventory
No extra abstractions needed	Cannot vary item price/effect per merchant easily
Easy to understand merchant-item linkage	Modifying relationships requires code changes

Design 2: Offer-based System

Pros	Cons
Decouples merchants from item implementation	More complex due to added abstraction
Supports different prices/effects per merchant	
Offers can be modified dynamically	
Adheres to Open-Closed Principle	

D. Weapon Sold Effects Implementation

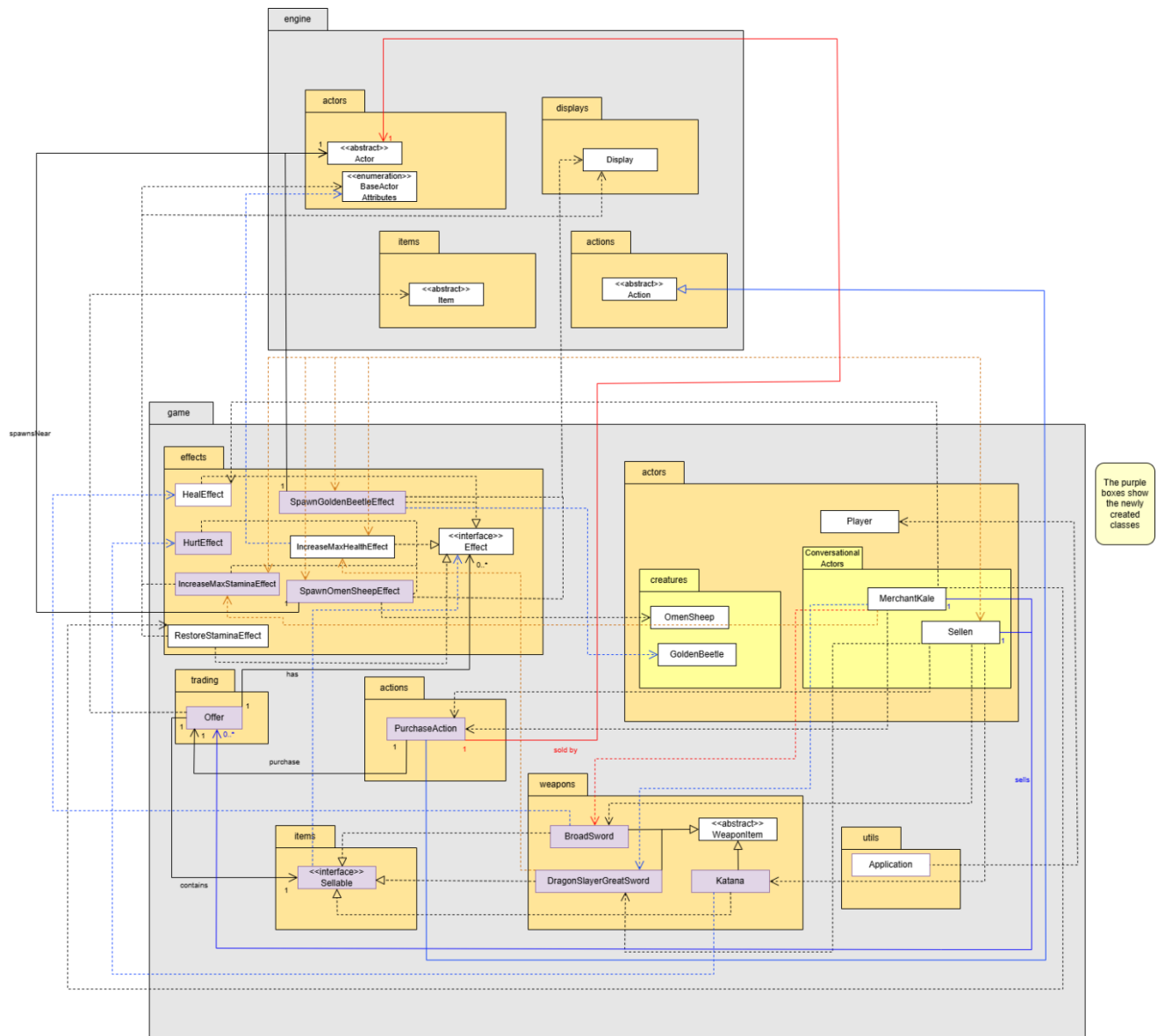
Design 1: Sellable Interface with a dedicated soldEffects method

Pros	Cons
Defines contract for sellable items	Adds an interface layer to the system
Enables polymorphism for saleable items	Requires implementation on all sellable items
Respects SRP by isolating sale behaviour	
Reduces duplication via centralized effect logic	

Design 2: Direct Effect Implementation in Merchant Classes

Pros	Cons
Simple design without interfaces	Tight coupling between merchants and effects
Direct effect control within merchant class	Effect logic duplicated across merchants
	Violates DRY principle with repeated logic

REQ4 - UML Diagram



The diagram above shows the final design which utilizes Design 1 for Merchant Implementation, Design 2 for Item Effects Management, Design 2 (Offer-based System) for Merchant-Item Relationship and Design 1 (Sellable Interface) for Weapon Sold Effects Implementation

The system design adheres strongly to the Single Responsibility Principle (SRP), ensuring that each component has a clear and focused purpose. The Effect interface handles effect application, the Offer class encapsulates trade details, and the Sellable interface defines item sale behaviour. This separation avoids overloading classes with multiple concerns.

The Open-Closed Principle (OCP) is followed by making the system extensible without requiring modification of existing code. We can create new effects via the Effect interface and add new sellable items through the Sellable interface. As an example, integrating a new effect like "SpawnSpiritGoatEffect" simply requires a new class that implements Effect, no existing classes need alteration. This modularity enables the system to evolve organically.

The Liskov Substitution Principle (LSP) is also respected. All effect classes comply with the Effect interface and sellable weapons correctly implement the Sellable interface. This ensures that any class using these interfaces can rely on consistent, expected behaviour regardless of specific implementations.

With the Interface Segregation Principle (ISP), the system avoids large, catch-all interfaces. For instance, the Sellable interface only has a single method called soldEffects() which exclusively defines behaviours of effects once an Item is sold.

The Dependency Inversion Principle (DIP) is applied by depending on abstractions rather than concrete implementations. Merchants interact with the Offer abstraction rather than directly with item classes, and purchase processes depend on the Effect interface instead of specific effects. For example, MerchantKale does not need to know the inner workings of the BroadSword or DragonSlayerGreatSword classes, but simply utilizes the Offer abstraction, enhancing flexibility and reducing coupling.

The system also follows the DRY (Don't Repeat Yourself) principle through centralized, reusable effect classes. Common functionalities, such as healing or stat modifications, are implemented once and reused wherever needed. Without this design, such logic would need to be duplicated across various weapon or merchant classes. The HealEffect class, for instance, can be used by both the Broadsword and the Katana without reimplementing the healing logic.

Connascence is minimized through clean interfaces and limited interdependencies. Purchase effects are agnostic to the internals of actors or merchants and rely solely on public interfaces. Weapons remain unaware of who sells them or at what price. Likewise, merchants do not require knowledge of how the effects in their offers are implemented. This decoupling reduces the risk of cascading failures when one part of the system changes.

In terms of future maintainability, the design is highly adaptable. Adding a new weapon requires only implementing the Sellable interface and extending the WeaponItem abstract class. Creating a new effect involves implementing the Effect interface. If merchant behavior changes, only the specific Merchant class needs updating. For instance, if a future feature demands weapons that

apply status effects upon purchase, a new `ApplyStatusEffect` class can be created and integrated without modifying any existing code. This new effect can be seamlessly composed with existing ones in various offers.

The decision to adopt an offer-based system with separate effect classes over a direct merchant-item binding model was made for its superior flexibility and extensibility. In a direct binding model, adding a merchant who sells existing items with unique effects would necessitate duplicating item logic. The offer-based design allows the same item to be sold by different merchants with distinct effects and prices by simply creating new `Offer` instances. This not only streamlines future development but also meets the key requirement that items like the Broadsword can be sold differently by Sellen and Kale, adapting easily to new gameplay needs.

REQ4 – Sequence Diagram

This sequence diagram shows the conditions required to initiate a BroadSword purchase interaction between the player and Merchant Kale.

